

Query for the SQL

1) Check total rows :-

```
-- Check total rows
SELECT COUNT(*) AS total_rows
FROM black_friday_data;
```

Output –

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
total_rows			
550068			

2) Check first 10 records:-

```
-- Check first 10 records
SELECT *
FROM black_friday_data
LIMIT 10;
```

Output:-

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category_1	Product_Category_2	Product_Category_3	Purchase
1000001	P00000142	F	0-17	10	A	2	0	3	4	5	13650
1000001	P00004842	F	0-17	10	A	2	0	3	4	12	13645
1000001	P00025442	F	0-17	10	A	2	0	1	2	9	15416
1000001	P00051442	F	0-17	10	A	2	2	8	17	Unknown	9938
1000001	P00051842	F	0-17	10	A	2	0	4	8	Unknown	2849
1000001	P00058142	F	0-17	10	A	2	0	3	4	12	11051

black_friday_data3

3) Check missing User_ID:-

```
-- Check missing User_ID  
SELECT COUNT(*) AS missing_user_id  
FROM black_friday_data  
WHERE User_ID IS NULL;
```

Output:-

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	missing_user_id			
▶	0			

4) Check missing Product_ID:-

```
SELECT COUNT(*) AS missing_product_id  
FROM black_friday_data  
WHERE Product_ID IS NULL;
```

Output:-

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	missing_product_id			
▶	0			

5) Check missing Purchase:-

```
SELECT COUNT(*) AS missing_purchase  
FROM black_friday_data  
WHERE Purchase IS NULL;
```

Output:-

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	missing_purchase			
▶	0			

6) Check invalid Purchase values:-

```
SELECT COUNT(*) AS invalid_purchase
FROM black_friday_data
WHERE Purchase <= 0;
```

Output:-

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	invalid_purchase			
▶	0			

7) Check duplicate transactions:-

```
SELECT User_ID, Product_ID, COUNT(*) AS duplicate_count
FROM black_friday_data
GROUP BY User_ID, Product_ID
HAVING COUNT(*) > 1;
```

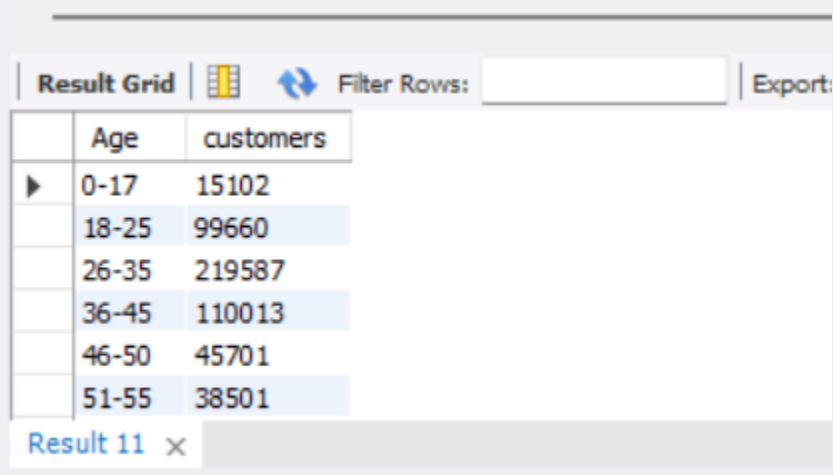
Output:-

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	User_ID	Product_ID	duplicate_count			

8) Check age categories:-

```
SELECT Age, COUNT(*) AS customers
FROM black_friday_data
GROUP BY Age
ORDER BY Age;
```

Output:-



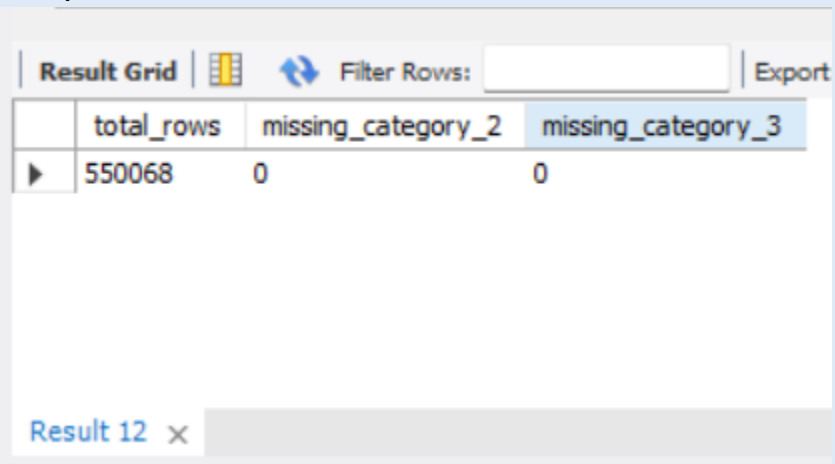
The screenshot shows a SQL query result in a 'Result Grid' window. The query is: `SELECT Age, COUNT(*) AS customers FROM black_friday_data GROUP BY Age ORDER BY Age;`. The result is a table with two columns: 'Age' and 'customers'. The data is as follows:

Age	customers
0-17	15102
18-25	99660
26-35	219587
36-45	110013
46-50	45701
51-55	38501

At the bottom of the window, it says 'Result 11' with a close button.

```
SELECT
    COUNT(*) AS total_rows,
    SUM(Product_Category_2 IS NULL) AS missing_category_2,
    SUM(Product_Category_3 IS NULL) AS missing_category_3
FROM black_friday_data;
```

Output:-



The screenshot shows a SQL query result in a 'Result Grid' window. The query is: `SELECT COUNT(*) AS total_rows, SUM(Product_Category_2 IS NULL) AS missing_category_2, SUM(Product_Category_3 IS NULL) AS missing_category_3 FROM black_friday_data;`. The result is a table with three columns: 'total_rows', 'missing_category_2', and 'missing_category_3'. The data is as follows:

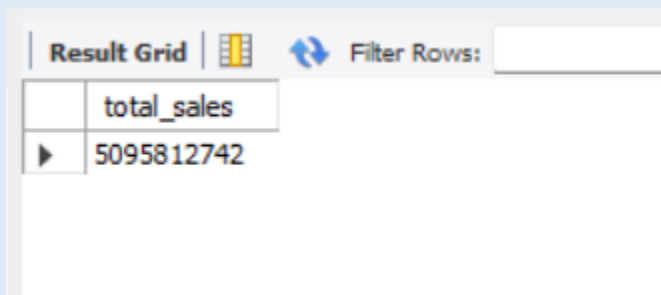
total_rows	missing_category_2	missing_category_3
550068	0	0

At the bottom of the window, it says 'Result 12' with a close button.

9) Analysis queries sql - total sales:-

```
SELECT  
    SUM(Purchase) AS total_sales  
FROM black_friday_data;
```

Output:-



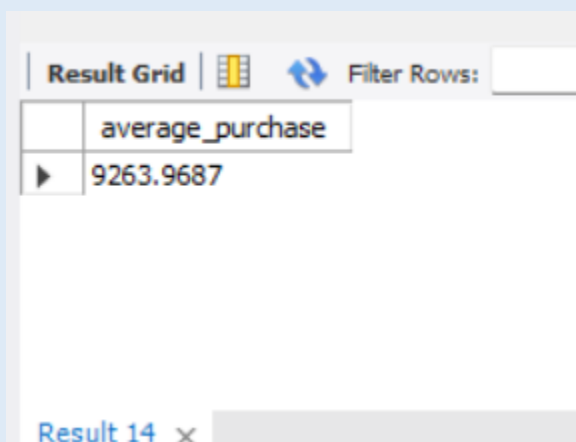
The screenshot shows a SQL query result grid. At the top, there are tabs for 'Result Grid' and a 'Filter Rows' input field. The table has one column named 'total_sales' and one row with the value '5095812742'.

	total_sales
▶	5095812742

10) Average purchase:-

```
SELECT  
    AVG(Purchase) AS average_purchase  
FROM black_friday_data;
```

Output:-



The screenshot shows a SQL query result grid. At the top, there are tabs for 'Result Grid' and a 'Filter Rows' input field. The table has one column named 'average_purchase' and one row with the value '9263.9687'. At the bottom left, there is a tab labeled 'Result 14' with a close button 'x'.

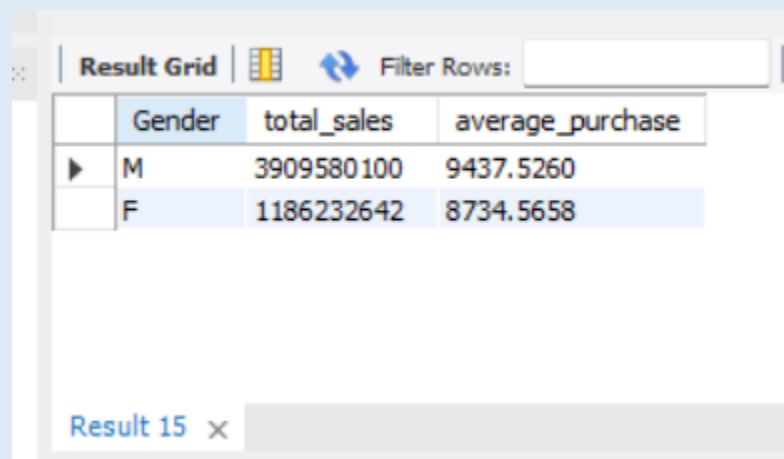
	average_purchase
▶	9263.9687

Result 14 x

11) Sales by gender:-

```
SELECT
    Gender,
    SUM(Purchase) AS total_sales,
    AVG(Purchase) AS average_purchase
FROM black_friday_data
GROUP BY Gender
ORDER BY total_sales DESC;
```

Output:-



The screenshot shows a SQL query result grid with the following data:

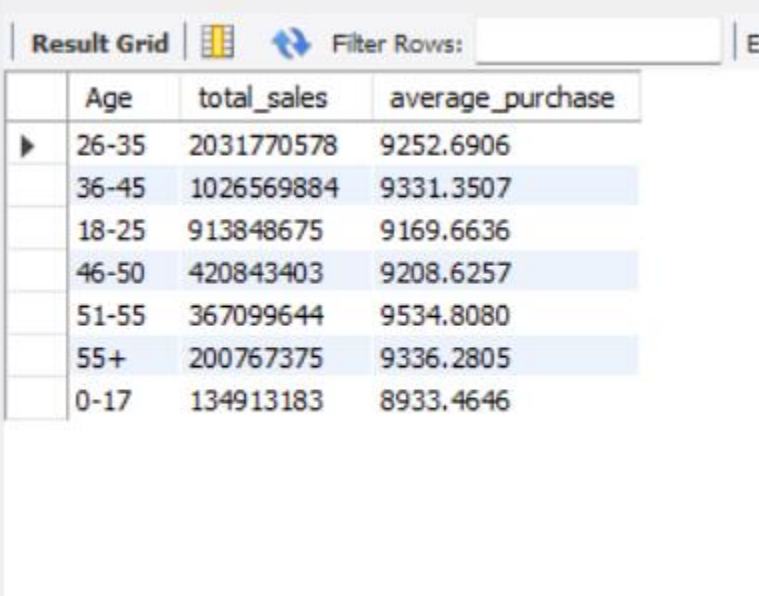
	Gender	total_sales	average_purchase
▶	M	3909580100	9437.5260
	F	1186232642	8734.5658

Result 15 x

12) Sales by age group:-

```
SELECT
    Age,
    SUM(Purchase) AS total_sales,
    AVG(Purchase) AS average_purchase
FROM black_friday_data
GROUP BY Age
ORDER BY total_sales DESC;
```

Output:-



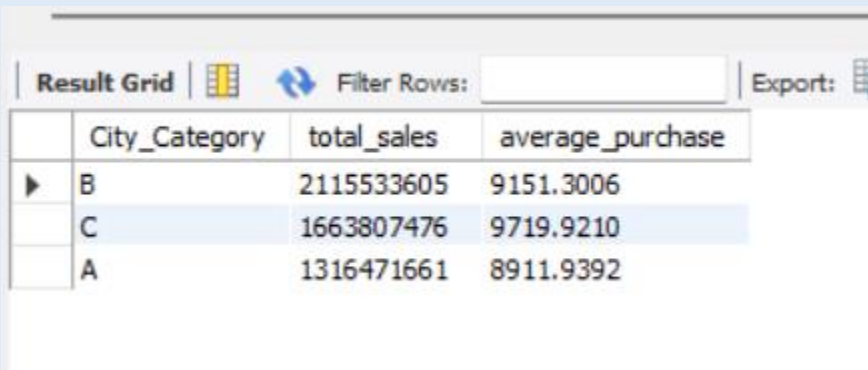
The screenshot shows a 'Result Grid' with a 'Filter Rows' input field. The table contains three columns: 'Age', 'total_sales', and 'average_purchase'. The data is sorted by 'total_sales' in descending order. The rows are as follows:

	Age	total_sales	average_purchase
▶	26-35	2031770578	9252.6906
	36-45	1026569884	9331.3507
	18-25	913848675	9169.6636
	46-50	420843403	9208.6257
	51-55	367099644	9534.8080
	55+	200767375	9336.2805
	0-17	134913183	8933.4646

13) Sales by city:-

```
SELECT
    City_Category,
    SUM(Purchase) AS total_sales,
    AVG(Purchase) AS average_purchase
FROM black_friday_data
GROUP BY City_Category
ORDER BY total_sales DESC;
```

Output:-



The screenshot shows a 'Result Grid' with a 'Filter Rows' input field and an 'Export' button. The table contains three columns: 'City_Category', 'total_sales', and 'average_purchase'. The data is sorted by 'total_sales' in descending order. The rows are as follows:

	City_Category	total_sales	average_purchase
▶	B	2115533605	9151.3006
	C	1663807476	9719.9210
	A	1316471661	8911.9392

14) Top 10 customers:-

```
SELECT
    User_ID,
    SUM(Purchase) AS total_spent
FROM black_friday_data
GROUP BY User_ID
ORDER BY total_spent DESC
LIMIT 10;
```

Output:-

Result Grid	Filter Rows:	Export:	Wrap Cell
User_ID	total_spent		
1004277	10536909		
1001680	8699596		
1002909	7577756		
1001941	6817493		
1000424	6573609		
1004448	6566245		
1005831	6512433		
1001015	6511314		
1003391	6477160		
1001181	6387961		

15) Best-selling product categories:-

```
SELECT
    Product_Category_1,
    COUNT(*) AS transactions,
    SUM(Purchase) AS total_sales
FROM black_friday_data
GROUP BY Product_Category_1
ORDER BY total_sales DESC;
```

Output:-

	Product_Category_1	transactions	total_sales
▶	1	140378	1910013754
	5	150933	941835229
	8	113925	854318799
	6	20466	324150302
	2	23864	268516186
	3	20213	204084713
	16	9828	145120612
	11	24287	113791115
	10	5125	100837301
	15	6290	92969042
	7	3721	60896731
	4	11753	27380488
	14	1523	20014696
	18	3125	9290201

Result 19 ×

16) Sales by occupation:-

```
# Sales by occupation
SELECT
    Occupation,
    COUNT(*) AS transactions,
    SUM(Purchase) AS total_sales,
    AVG(Purchase) AS average_purchase
FROM black_friday_data
GROUP BY Occupation
ORDER BY total_sales DESC;
```

Output:-

Result Grid  Filter Rows: Export:  Wrap Cell Content: 				
	Occupation	transactions	total_sales	average_purchase
▶	4	72308	666244484	9213.9803
	0	69638	635406958	9124.4286
	7	59133	557371587	9425.7282
	1	47426	424614144	8953.1933
	17	40043	393281453	9821.4782
	12	31179	305449446	9796.6402

Result 2 x 

SQL ANALYSIS

- Total Sales
- Average Purchase
- Customer Analysis
- Gender Analysis
- Age Analysis
- City Analysis
- Occupation Analysis
- Product Analysis
- Category Analysis
- Repeat Customers